

Practical 1:

Part B: Analysis Questions

Q1. How many objects are created?

Answer:

Two objects are created because the constructor is called twice using the new keyword.

Q2. Why does the comparison return false?

```
System.out.println(config1 == config2);
```

Answer:

The == operator compares object references. Since config1 and config2 refer to two different objects created with new, the result is false.

Q3. What problems can occur if configuration objects are created repeatedly?

Answer:

- Unnecessary memory usage.
- Inconsistent configuration values if objects are modified separately.
- Increased maintenance complexity.
- Reduced application performance.

Q4. Is creating multiple configuration objects necessary in this scenario? Why?

Answer:

No. A configuration class stores common application settings that should remain the same throughout the system. Therefore, only one shared instance is required.

Q5. Which software quality attributes are affected?

Answer:

- Memory Efficiency
- Performance
- Maintainability

- Consistency
- Reliability

Part C: Problems in the Existing Design

1. Multiple configuration objects can be created unnecessarily.
2. Memory is wasted by storing duplicate configuration data.
3. Configuration values may become inconsistent if different objects are modified independently.

Practical 2 :

Part B: Analysis Questions

Q1. How does the program decide which notification type to send?

Answer:

The program checks the notification type using multiple if-else statements and sends the corresponding notification.

Q2. Why are multiple if-else statements considered a bad design?

Answer:

They make the code harder to maintain, less readable, and require modifications whenever a new notification type is added.

Q3. What problems can occur if notification creation logic is written repeatedly?

Answer:

- Code duplication
- Difficult maintenance
- Increased chance of bugs
- Inconsistent implementation

Q4. Is centralizing object creation necessary in this scenario? Why?

Answer:

Yes. Centralizing object creation improves maintainability, reduces duplication, and makes the system easier to extend.

Q5. Which software quality attributes are affected?

Answer:

- Maintainability
- Reusability
- Scalability
- Flexibility
- Readability

Part C: Problems in the Existing Design

1. Multiple if-else statements make the code difficult to maintain.
2. Adding new notification types requires modifying existing code.
3. Object creation is tightly coupled with business logic.

Practical 3: Builder Pattern

Part B: Analysis Questions

Q1. How is the Student object created in the current implementation?

Answer: It is created using a constructor with many parameters.

Q2. Why is using a constructor with many parameters considered a bad design?

Answer: It is difficult to read, understand, and remember the correct parameter order.

Q3. What problems can occur if constructors have many parameters?

Answer: It increases the chance of passing incorrect values and makes the code harder to maintain.

Q4. Is using the Builder Pattern necessary in this scenario? Why?

Answer: Yes. It makes object creation simpler, clearer, and more flexible.

Q5. Which software quality attributes are affected?

Answer: Readability, maintainability, flexibility, and reliability.

Part C: Problems in the Existing Design

1. Constructor has too many parameters.
2. Easy to pass parameters in the wrong order.
3. Code is difficult to read and maintain.